

Thread Implementations: <u>Process and Thread</u> A single process can have multiple threads (multithreaded process) Thread shares: - Memory context: Text, Data, Heap - OS Context: Process ID, files Unique: - Identification (usually thread id) - Registers (General purpose and special) - Stack (not exactly true, divide stack space) Threads Benefits: - Economy - Require less resources to manage <u>User Thread</u> - Implemented as a user library - Runtime system (in the process) will handle thread related operation - Kernel is not aware of the threads in the process Pros: > Can have multithreaded program on ANY OS > Thread operations are just library calls > Generally more configurable and flexible - (Customised thread scheduling policy) Cons: > OS not aware of threads, scheduling is performed at process level - One thread blocked → Process blocked → All thread blocked <u>Kernel Thread</u> - Process table and thread table - Thread is implemented in the OS - Thread operation is handled as system calls - Thread-level scheduling is possible: - Kernel schedule by threads, instead of by process - Kernel may make use of threads for its own execution Pros: > Kernel can schedule on thread levels: - More than 1 thread in the same process can run simultaneously on multiple CPUs. True parallelism Cons: > Thread operations is now a system call - Slower and more resource intensive > Less flexible: - Used by all multithreaded program - Not flexible for some programs - Overkill for some programs <u>Hybrid Thread Model</u> - Have both - OS schedule on kernel threads only - User thread can binds to a kernel threads <u>Threads on Modern Processor</u> - Started off as a software mechanism - Hardware support - Multiple set of registers to allow threads to run natively and in parallel on the same core (simultaneous multi-threading)					
Inter-Process Communication IPC - Hard for cooperating processes to share information - Memory space is independent - Inter-Process Communication mechanisms (IPC) is needed Common IPC Mechanisms: <u>Shared-Memory</u> - Create memory region and attaches to memory region (Switching to kernel mode only when creating and attaching) Pros: - Efficient - OS calls only for create and attach - Ease of use Cons: - Cannot synchronise - Harder implementation (use of virtual memory system) <u>Message Passing</u> Stored in kernel memory Process send message to another process (system calls) Communication- identify other process: - Direct (send(destination, msg), receive(source, msg)) - Indirect/ Mailbox/ port (send(mailbox, msg), receive(mailbox, msg)) Synchronisation: - Synchronous - Asynchronous					
<table><tr><th>Pros</th><th>Cons</th></tr><tr><td>Portable Easier synchronisation</td><td>Inefficient (kernel calls) Harder to use</td></tr></table>	Pros	Cons	Portable Easier synchronisation	Inefficient (kernel calls) Harder to use	
Pros	Cons				
Portable Easier synchronisation	Inefficient (kernel calls) Harder to use				

Unix-specific: <u>Pipe</u> - One of the earliest IPC mechanism - General Idea: A communication channel is created with 2 ends: Write into and Read from - Character oriented - Unix shell uses " " to link input/output channels of one process to another, known as piping - Pipe is stored in a file Communication channel: stdin, stdout, stderr *stdout can be redirected to a file but stderr cannot As IPC Mechanism - A pipe can be shared between two processes - A form of Producer-Consumer relationship - Close end that not used to get the correct signals - It is possible to - > Attach/change the standard communication channels (stdin, stdout, stderr) to one of the pipes - > Redirect the input/output from one program to another - > dup() - creates a copy of a FD and assigns to a random FD - > dup2() - creates a copy of a FD and assigns to another FD <u>Unix Signal</u> - An asynchronous notification regarding an event sent to a process/thread - The recipient of the signal must handle the signal by: - A default set of handlers OR - User supplied handler (only for some signals) Problems with Concurrent Execution: - When two or more processes: - Execute concurrently in interleaving fashion - Share a modifiable resource - Execution of a single sequential process is deterministic - Repeated execution gives the same result - Execution of concurrent processes may be non-deterministic - Execution outcome depends on the order in which the shared resource is accessed/modified - known as race conditions <u>Race Condition</u> Incorrect execution is due to the unsynchronized access to a shared modifiable resource General outline of solution: - Designate code segment with race condition as critical section - At any point in time, only one process can execute in the critical section - Other process are prevented from entering the same critical section <u>Critical Section:</u> - Use of shared resource Properties of Correct CS Implementation: Mutual Exclusion: - If process P _i is executing in critical section, all other processes are prevented from entering the critical section. Progress: - If no process is in a critical section, one of the waiting processes should be granted access. Bounded Wait: - After process P _i request to enter critical section, there exists an upperbound of number of times other processes can enter the critical section before P _i . Will not wait infinitely. Independence: - Process not executing in critical section should never block other process. <u>Symptoms of Incorrect Synchronization</u> Deadlock: Requires Mutual Exclusion, Hold and Wait, No pre-emption, Circular Wait - All processes blocked → no progress Livelock: - Usually related to deadlock avoidance mechanism - Processes keep changing state to avoid deadlock and make no other progress - Typically process are not blocked - Starvation: - Some processes are blocked forever <u>Implementation of Critical Section</u> - Assembly level implementations: - Mechanisms provided by the processor - Load Link/ Store Conditional - Test and Set - High level language implementations: - Utilizes only normal programming constructs - High level abstraction: - Provide abstracted mechanisms that provide additional useful features - Commonly implemented by assembly level mechanisms	
---	--

High Level Language Implementation <u>Peterson's Algorithm:</u> Disadvantages: Busy Waiting: The waiting process repeatedly test the while loop condition instead of going into blocked state Low level: Higher-level programming construct is desirable -simplify mutual exclusion -less error prone Not general: General synchronization mechanism is desirable, not just mutual exclusion Memory Abstraction <u>Memory Hardware</u> • Random Access Memory (RAM) - Each byte unique index (physical address) • Contiguous memory region Transient Data: Data that only exist during execution Persistent Data: Data that outlives the process OS handle memory task • Allocate memory space to new process • Manage memory space for process • Protect memory space of processes from each other • Provide memory related system calls to process • Manage memory space for internal use <u>Without Memory Abstraction (use physical address)</u> • ☒ Directly at specified address • ☒ Hard to protect memory space <u>Address Relocation:</u> • ☒ Slow loading time (Recalculate during loading) • ☒ Not easy to distinguish memory reference from normal integer constant <u>Base + Limit Solution:</u> • Use special register as Base Register and Limit Register • ☒ Incur cost to check limit • ☒ Generalized to segmentation mechanism • ☒ Provides a crude memory abstraction <u>Memory Abstraction : Logical Address</u> <u>Contiguous Memory Management</u> Assumptions: 1. Each process occupies a contiguous memory region 2. The physical memory is large enough to contain one or more processes with complete memory space Memory Partition scheme Fixed Sized Partition Physical memory split into fixed number of partitions ☒ Easy to manage and fast to allocate ☒ Internal fragmentation, need to be large to contain large processes. Variable-Size Partition (Dynamic Partitioning) Partition is created based on the actual size of process OS keep track of the occupied and free memory regions ☒ Flexible and remove internal fragmentation ☒ External fragmentation ☒ Require more time to locate regions ☒ Need to maintain more information in OS <u>Allocation Algorithms</u> o First-Fit – Take first large enough hole o Best-Fit – Find the smallest large enough hole o Worst-Fit – Find the largest hole o Buddy System - Find Buddy -> size S, Sth bit are complements Compaction - Move occupied partition around to consolidate holes ☒ Fast Allocation ☒ Still has some internal fragmentation	
Disjoint Memory Schemes <u>Paging</u> • Physical memory split into physical frame • Logical memory split into logical page • At execution, pages loaded into any available memory frame • Logical memory space remain contiguous • Occupied physical memory region can be disjoint • Page Table: Lookup Mechanism • Maps logical page to physical frame • PA = F x size of physical frame + Offset • Page/Frame size of 2 ⁿ (n bits offset) • M bits of logical address ☒ No external fragmentation ☒ Clear separation of logical and physical address space • Great flexibility and simple address translation ☒ Internal fragmentation only on last frame	

Implementation • Pure-software solution • OS stores page table information in PCB • Improved understanding: o Memory context of a process == Page table ☒ Require 2 memory access for every memory reference (page table and frame) • Hardware Support • Translation Look-Aside Buffer (TLB) in modern processors o Act as a cache for a few page table entries • Logical address translation with TLB o Use page number to search o Entry found (TLB Hit) • Retrieve frame number to get physical address o Entry not found (TLB Miss) • Memory access to access full page table • Retrieve frame number to get physical address and update TLB • Part of Hardware Context of a process • TLB entries are flushed when context switch • Will encounter many TLB miss when resume (possible to save some entries, use PID) • Protection: • Access-Right Bits • Each PTE has (writable, readable, executable) bits • Valid Bit – Indicate page is valid to access • OS set valid bits when process is running • Memory access is checked against this (Out of range access) • Page sharing: • Page table can allow several processes to share the same physical memory frame • Usage • Shared code page (code used by many processes eg library) • Implement Copy-On-Write o Parent and child share a page until one changes value <u>Segmentation</u> • Separate regions into multiple memory segments • Each segment has name(usually a number) and limit • Memory reference: Segment name + Offset • Segment table: • Each segment mapped to a contiguous physical memory • SegID is used to look up <Base, Limit> of a segment • PA = Base + Offset • Offset < Limit for valid access ☒ Each segment is an independent contiguous memory space • Can grow/shrink independently • Can be protected/shared independently ☒ External fragmentation due to variable size contiguous memory regions <u>Segmentation + Paging</u> o Each segment has its own page table o Lookup SegmentID to get page number o -> Use page number to get frame number o -> Use frame number to get physical address	
Virtual Memory Management Virtual Memory • Secondary Storage Capacity >> Physical Memory Capacity • Split logical address space, some in physical memory some on secondary storage • Extended Paging Scheme o Page Type (use a bit in PTE) • Memory resident – in physical memory • Non-memory resident – in secondary storage o CPU can only access memory resident pages • Page Fault: When CPU access non-memory resident • OS needs to bring a non-memory resident page into physical memory o Accessing Page • Check page table • Is page X memory resident? • Yes: Access physical memory location • No: Continue to next step • Page Fault: Trap to OS • Locate page X in secondary storage • Load page X into physical memory • Update page table • Go back to step 1 to retry ☒ If memory access always page fault == Thrashing	

Locality Principles • Temporal locality • Memory address which is used is likely to be used again • After a page is loaded into physical memory, likely to be accessed again • Spatial locality • Memory addresses close to a used address is likely to be used • A page contains contiguous locations that is likely to be accessed again <u>Demand Paging</u> • Process starts with no memory resident page • Only load a page when there is a page fault • ☒ Fast startup time for new process • ☒ Small memory footprint • ☒ Process appear sluggish at start due to many page faults • ☒ Page fault may have cascading effect on other processes (thrashing) Problems: • Huge page table with large logical memory space o How to structure the page table for efficiency? o Page Table Structures • Each process has limited number of resident memory pages o Which page should be replaced when needed? o Page Replacement Algorithms • Limited physical memory frames o How to distribute among the processes? o Frame Allocation Policies Page Table Structure • Page table info is kept with process info, takes up physical memory space • Modern computer system provide huge logical memory space o 4GiB norm, 8TiB or more is possible o Huge logical memory space-> Huge number of pages o Each page has a PTE -> Huge page table • Problem with huge page table: o High overhead (for small processes) o Fragmented page table ▪ Page table occupies several memory pages Page Table Structure: Direct Paging • Keep all entries in a single table • 2 ⁿ pages in logical memory space • P bits to specify one unique page • 2 ⁿ PTE with physical frame number + additional information bits Page Table Structure: 2 – Level Paging • Process may not use the entire virtual memory space • Split full page table into smaller page tables • Original 2 ⁿ entries • With 2 ^m smaller page tables, m bits to identify one page table (in page directory) • Each smaller page table contains 2 ⁿ (p-m) entries • ☒ Huge space saving per process (especially small ones) Inverted Page Table • Keep a single mapping of physical frame to <pid, page#> • Page# not unique among processes • ☒ Huge saving: One table for all processes • ☒ Slow translation, lookup page require search of whole table Page Replacement Algorithms: • No free physical memory frame during a page fault • Need to evict a memory page • When a page is evicted: o Clean page: not modified -> no need to write back o Dirty page: modified -> need to write back • Modeling Memory References: o Modeled as memory reference strings, using page numbers • Evaluation: o T(access) = (1-p) * T(mem) + p * T(pagefault) o Reduce p to keep access time reasonable since T(pf) >> T(mem) <u>Optimum (OPT)</u> • Replace page that will not be used again for the longest period of time (Next use time) • Guarantees minimum number of page faults • ☒ Require future knowledge, not realizable <u>FIFO</u> • OS maintain a queue of resident page number o Remove the first page in queue if replacement is needed o Update the queue during page fault trap ☒ Simple to implement- no hardware support needed ☒ Belady's Anomaly – increasing no. of frames, increase page fault → Not exploiting temporal locality	
---	--

Least Recently Used (LRU)

- Make use of temporal locality (**Last Used Time**)
 - Replace page that has not been used for the longest time
 - Expect page to be reused in a short time window
 - Attempts to approximate OPT algorithm
 - Does not suffer from Belady's Anomaly
- Implementation: Need to keep track of "last access time" (hardware support)

Approach A – Counter

- A logical time counter that increment for every memory reference
 - PTE has a "time of use" field
 - Store time counter value whenever referenced
 - Replace page with smallest
 - Need to search through all pages
 - "Time-of-use" is forever increasing (overflow)

Approach B – "Stack"

- Maintain a stack of page numbers
- If page X is referenced
 - Remove from the stack (for existing entry)
 - Push on top of stack
- Replace the page at the bottom of stack
- No need to search through all entries
- Not a pure stack: Entry can be removed anywhere in the stack
- Hard to implement in hardware

Second-Chance (Clock)

- Modified FIFO to give second chance for accessed page
- Each PTE maintains a "reference bit" (1 = Accessed, 0 = Not Accessed)
- Algorithm:
 - The oldest FIFO page is selected
 - If reference bit == 0 -> Page is replaced
 - If reference bit == 1 -> Page is given 2nd chance
 - Reference bit cleared to 0
 - Next FIFO page is selected, continue next
- When all pages has reference bit == 1 (becomes FIFO)

Frame Allocation:

- N physical memory frames, M processes
- Equal Allocation - Each process get M/N frames
- Proportional Allocation
 - Processes are different in size (memory usage)
 - Each process get (size of process / total size of all processes)* N frames
- Local Replacement (Kick out page from this process) assumed for PRA
 - Frames allocated to a process remain constant
 - Performance is stable between multiple runs
 - Badly behaved process can affect others
 - Hinder the progress of a process

Global Replacement (choose page to kick from ALL frames)

- Allow self-adjustment between processes
 - Process that needs more frames can get from other processes
- Badly behaved process can affect others
- Frames allocated to a process can be different from run to run

Thrashing

- Insufficient physical frames -> Thrashing
- Heavy I/O to bring non-resident pages into RAM
- Hard to find the right number of frames
 - If global replacement is used
 - A thrashing process "steals" page from other process
 - Causes other process to thrash (Cascading Thrashing)
 - If local replacement is used
 - Thrashing can be limited to one process
 - Single process may hog I/O and degrade performance of other processes
 - Set of pages referenced by a process is relatively constant in a period of time (locality)

Working Set Model

- A process will cause page fault for the set of pages in a new locality
- No/few page faults until process transits to new locality
- Defines Working Set Window Δ (an interval of time)
- W(t, Δ) = active pages in the interval at time t
- Allocate enough frames for pages in W(t, Δ) to reduce possibility of page fault
- Transient region – working set changing in size
- Stable region – working set about the same for a long time
- Accuracy of working set model is affected by choice of Δ(time window)
 - Too small: May miss pages in the current locality
 - Too big: May contain pages from a different locality

File Systems Introduction

- Self-Contained
 - Information stored on media is enough to describe entire organization
- Persistent
 - Beyond the lifetime of OS and processes
- Efficient
 - Provides good management of free and used space
 - Minimum overhead for bookkeeping information

File System Abstraction

File

- Represent a logical unit of information created by process
 - Abstract Data Type
 - A set of common operations with various possible implementations
- Contains
 - Data: Information structured in some way
 - Metadata: Additional information (file attributes)
- File Protection
 - Restrict access based on the user identity (common)
- Access Control List
 - A list of user identity and the allowed access types
 - Very customisable
 - Too much information associated with file
 - Classify users into 3 class
 - Owner – user who created the file
 - Group – Set of users who need similar access to a file
 - Universe – All other users in the system
- Operations on File Metadata
 - Rename
 - Change attributes
 - Read attribute

File Data: Structure

- Array of bytes – each byte with unique offset from file start
- Fixed length records – Array of records, can grow/shrink, can jump to any record easily
- Variable length records – Flexible but harder to locate a record

File Data: Access Methods

- Sequential Access
 - Data read in order, starting from the beginning
 - Cannot skip but can be rewound
- Random Access
 - Data can be read in any order
 - Data can be provided in two ways
 - Read(Offset): explicitly state position to be accessed
 - Seek(Offset): a special operation is given to move to a new location in file (Unix and windows)
- Direct Access
 - Used for file containing fixed length records
 - Allow random access to any record directly

File Operations as System Calls

- OS provides file operations as system calls
- Information kept for an opened file
 - File Pointer: current location in file
 - Disk Location: actual file location on disk
 - Open Count: number of process opened this file

System-wide open-file table

- One entry per unique file
- Per process open-file table
 - One entry per file used in the process
 - Each entry points to the system-wide table

Directory

- Provide a logical grouping of file – user view of directory
- Keep track of files – actual system usage of directory
- Ways to structure a directory
 - Single-level
 - Tree-Structure
 - Absolute Pathname
 - Relative Pathname
 - Directed Acyclic Graph (DAG)
 - If a file can be shared
 - One copy of actual content
 - "Appear" in multiple directories with different path names
 - Two implementations in Unix
 - Hard link – limited to file only
 - low overhead, only pointers added
 - deletion problems
 - Symbolic Link – can be file or directory
 - Creates a special link file contain path name of F
 - When G is accessed: find F, access F
 - Simple deletion

- Larger overhead, takes up disk space
- General Graph
 - not desirable – hard to traverse (possible to form cycles)

File System Implementation

- Disk Structure
 - 1-D array of logical blocks
 - Logical block – Smallest accessible unit (usually 512bytes to 4KB)
 - Logical block is mapped into disk sector(s)
 - Layout of disk sector is hardware dependent
- Disk Organization
 - Master Boot Record (MBR) at sector 0 with partition table
 - Simple boot code & partition table
 - Followed by one or more partitions
 - Each partition can contain an independent file system
 - A file system generally contains
 - OS Boot-Up information
 - Partition details
 - Total number of blocks
 - Number and location of free disk blocks
 - Directory Structure
 - File Information
 - Actual File Data

File Implementation

- A collection of logical blocks
- File size != multiple of logical blocks -> internal fragmentation last block
- A good file implementation must
 - Keep track of the logical blocks
 - Allow efficient access
 - Disk space is utilized effectively

File Block Allocation 1: Contiguous

- Allocate consecutive disk blocks to a file
 - Simple to keep track and Fast access (find first block)
- External Fragmentation
 - File size need to be specified in advance

File Block Allocation 2: Linked List

- Keep a linked list of disk blocks
- Each stores
 - The next disk block number
 - Actual file data
- File information stores first and last disk block no.
 - Solve fragmentation problem
 - Random access in a file is very slow
 - Part of disk block is used for pointer
 - Less reliable (pointer incorrect)

File Block Allocation 2: Linked List v2

- Move block pointer into a table
 - File Allocation Table(FAT)
 - FAT in memory at all times
- Simple yet efficient (used by MS-DOS)
- Faster random access -> traversal in mem
- FAT keep tracks of all disk blocks in a partition
 - Can be huge when disk is large
 - Consumes valuable memory space

File Block Allocation 3: Indexed Allocation (I-Node)

- Each file has an index block
 - An array of disk block addresses
 - IndexBlock[N] == Nth Block address
- Less memory overhead – only open file's index block need in memory
- Fast direct access
- Limited maximum file size
 - Max number of blocks == Number of index block entries
- Index block overhead
- Larger variation
 - Linked scheme: linked list of index blocks
 - Multilevel index: similar to multi-level paging
 - Combination

Free Space Management

- Need to know free blocks to allocate file
- Bitmap
 - Each disk block represented by 1 bit
 - Provide a good set of manipulations
 - Need to keep in memory for efficiency reason
- Linked List
 - Each disk block contains
 - A number of free disk block numbers
 - A pointer to the next free space disk block

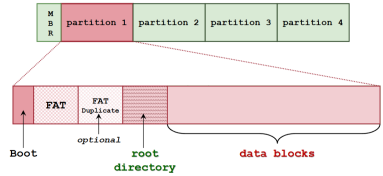
- Easy to locate free block
- Only first pointer needed in memory
- High overhead

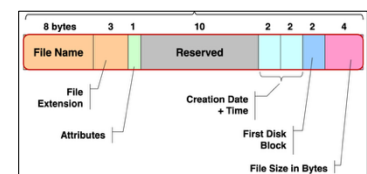
Directory Structure

- Keep track of file in directory
- Map file name to file information
- Directory Implementation: Linear List
 - Each entry represent a file
 - Store file name (minimum) and possibly other metadata
 - Store file information or pointer to file information
 - Linear search inefficient – can be cached
- Directory Implementation: Hash Table
 - Usually chained collision
 - Fast Lookup
 - Limited size, need good hash function
- Directory Implementation: File Information
 - Can be stored
 - Everything in directory entry
 - Store only file name and pointers to other some info (data struct)
- File Operation
 - Create -
 - Locate parent directory and search for file name
 - If found terminate with error
 - Use free space list to find free disk block
 - Add an entry to parent directory
 - Open – check system wide table first

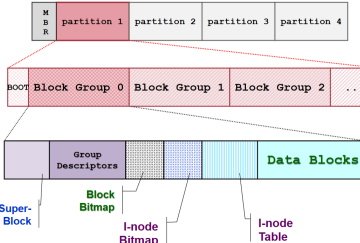
File System Case Studies

Microsoft FAT File System

- 
- FAT12 -> FAT16 -> FAT32
 - Support larger hard disk as a single partition
 - Disk Cluster
 - Instead of using a single disk block as the smallest allocation unit -> use a number of contiguous disk blocks
 - FAT Size
 - Bigger FAT -> More disk block/cluster -> More bits to represent
 - FAT
 - One entry per data block/cluster
 - Store disk block information
 - OS cache in RAM to facilitate linked list traversal
 - FREE, Block number, EOF code, BAD code
 - Directory Structure
 - Special type of file
 - File Name + Extension
 - Limited to 8 + 3 characters
 - The first byte of file name may have special meaning
 - File Creation time and date
 - Year limited to 1980 – 2107
 - Accuracy of second is + 2s
 - First Disk Block Index
 - 12 bits for FAT12
 - Cluster Size
 - Larger cluster size -> Larger usable partition
 - Larger cluster size -> Larger internal fragmentation
 - On FAT 32, 32-bit sector count -> limited partition size
 - Only 28 bit is used in disk block/cluster index
 - Long File Name Support
 - Virtual FAT
 - Support up to 255 characters
 - Workaround
 - Use multiple directory entries for a file with long file name
 - 8+3 for backwards compatibility



Extended-2 File System (Linux)

- 
- Superblock
 - Describe the whole file system
 - Total I-Node number, I-nodes per group, Total disk blocks, Disk blocks per group, etc
 - Duplicated in each block group for redundancy
 - Group descriptors
 - Describe each of the block group
 - Number of free disk blocks, free I-nodes
 - Location of the bitmaps
 - Duplicated in each block group
 - Block Bitmap
 - Status of blocks in this block group
 - I-Node Bitmap
 - Usage status of I-nodes of this block group
 - I-Node table
 - An array of I-Node (Access by an unique index)
 - Contains only I-Nodes of this block group
 - I-Node Data Blocks
 - Other than file metadata, I-Node contains 15 disk block pointers(disk block numbers) 128 bits
 - First 12 pointers (direct blocks)
 - 13th Pointer (Single Indirect Block), 14th double, 15th triple
 - Fast access to small file
 - Flexibility in handling large files
 - Directory Structure
 - Data blocks of a directory stores
 - A linked list of directory entries for file/subdirectories information within this directory
 - Each directory entry contains
 - I-node number for that file/subdirectory
 - Size of this directory entry (to locate next entry)
 - Length of the file/subdirectory name
 - Type: File or Subdirectory or others
 - File/Subdirectory name (up to 255 characters)
 - Hard/Symbolic Link with I-Node
 - Hard link
 - Point to the same I-Node
 - Reference Count in I-Node increase, decrease for deletion
 - Symbolic link
 - Creates a I-Node that points to file
 - Only pathname is stored, link easily invalidated (change name)

